NLP portion

In [1]:
```
import os, subprocess

REPO_URL = "https://github.com/tongyuguo/HelpHerInvest.git"
REPO_DIR = "HelpHerInvest"
data_dir = os.path.join(REPO_DIR, "Data")

def clone_or_pull():
    if os.path.isdir(os.path.join(REPO_DIR, ".git")):
        subprocess.run(["git", "-C", REPO_DIR, "pull"])
    else:
        subprocess.run(["git", "clone", REPO_URL])

clone_or_pull()
```

```
error: Pulling is not possible because you have unmerged files.
hint: Fix them up in the work tree, and then use 'git add/rm <file>'
hint: as appropriate to mark resolution and make a commit.
fatal: Exiting because of an unresolved conflict.
```

In [2]:
```
import pandas as pd
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
#%pip install sentence-transformers
from sentence_transformers import SentenceTransformer
import zipfile

STOCKS_CSV = "HelpHerInvest/Data/nlp_clean_stock_symbols.csv"

def load_stocks(path=STOCKS_CSV):
    df = pd.read_csv(path)

    # Choose a text field for retrieval
    df["kw_text"] = df["document_clean_tfidf"].fillna(df["document_raw
    df["sem_text"] = df["document_raw"].fillna(df["document_clean_tfid

    # Keep only usable rows
    df = df[(df["kw_text"].str.len() > 0) & (df["sem_text"].str.len()
    return df

def build_hybrid_index(df,
                       sbert_model="sentence-transformers/all-MiniLM-L
                       tfidf_min_df=2,
                       tfidf_ngram_range=(1,2)):
    # TF-IDF for keyword similarity
    vectorizer = TfidfVectorizer(min_df=tfidf_min_df, max_df=0.95, ngr
    X_tfidf = vectorizer.fit_transform(df["kw_text"].astype(str))

    # SBERT embeddings for semantic similarity
```

```
    model = SentenceTransformer(sbert_model)
    X_emb = model.encode(
        df["sem_text"].astype(str).tolist(),
        batch_size=128,
        show_progress_bar=True,
        normalize_embeddings=True
    ).astype(np.float32)

    return vectorizer, X_tfidf, model, X_emb

df_stocks = load_stocks(STOCKS_CSV)
vectorizer, X_tfidf, sbert, X_emb = build_hybrid_index(df_stocks)
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please
set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights:   0%|          | 0/103 [00:00<?, ?it/s]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/archit
ecture; not ok if you expect identical arch.
Batches:   0%|          | 0/81 [00:00<?, ?it/s]
```

In [3]:

```
import numpy as np

def _minmax(x):
    x = np.asarray(x, dtype=np.float32)
    mn, mx = x.min(), x.max()
    return (x - mn) / (mx - mn + 1e-9)

def build_query_from_df_text_inputs_row(row):
    # Use text_input if present; otherwise combine topics
    t = str(row.get("text_input", "")).strip()
    if t:
        return t
    p = str(row.get("primary_topic", "")).strip()
    s = str(row.get("secondary_topic", "")).strip()
    return (p + " " + s).strip()

def hybrid_retrieve(query, df_stocks, vectorizer, X_tfidf, sbert, X_em
                    top_k=50,
                    w_sem=0.65, w_kw=0.35,
                    mmr_lambda=0.80,
                    candidate_pool=500,
                    sector_cap=10):
    """
    w_sem/w_kw: weight semantic vs keyword similarity
    mmr_lambda: higher => more relevance, lower => more diversity
    candidate_pool: retrieve this many by relevance before diversifyin
```

```
    sector_cap: max results per sector (set None to disable)
    """

    # keyword similarity (TF-IDF cosine for L2-normalized TF-IDF is do
    q_tfidf = vectorizer.transform([query])
    kw_scores = (X_tfidf @ q_tfidf.T).toarray().ravel()

    # semantic similarity (cosine via normalized vectors)
    q_emb = sbert.encode([query], normalize_embeddings=True).astype(np
    sem_scores = (X_emb @ q_emb.T).ravel()

    # combine (normalize each so neither dominates)
    kw_n = _minmax(kw_scores)
    sem_n = _minmax(sem_scores)
    scores = w_sem * sem_n + w_kw * kw_n

    # candidate set
    pool = min(candidate_pool, len(df_stocks))
    cand_idx = np.argpartition(-scores, pool-1)[:pool]
    cand_idx = cand_idx[np.argsort(-scores[cand_idx])]

    # MMR diversification using embedding similarity to reduce redunda
    selected = []
    selected_set = set()

    # Precompute candidate embeddings
    cand_emb = X_emb[cand_idx]

    # Sector tracking
    sector_counts = {}
    def sector_ok(i):
        if sector_cap is None:
            return True
        sec = df_stocks.loc[i, "sector"]
        sec = sec if isinstance(sec, str) and sec else "Unknown"
        return sector_counts.get(sec, 0) < sector_cap

    for _ in range(min(top_k, len(cand_idx))):
        best_i = None
        best_mmr = -1e9

        for pos, i in enumerate(cand_idx):
            if i in selected_set:
                continue
            if not sector_ok(i):
                continue

            rel = scores[i]

            if not selected:
                mmr = rel
            else:
```

```
                # max similarity to already selected (redundancy)
                sim_to_selected = np.max(X_emb[i] @ X_emb[selected].T)
                mmr = mmr_lambda * rel - (1 - mmr_lambda) * sim_to_sel

            if mmr > best_mmr:
                best_mmr = mmr
                best_i = i

        if best_i is None:
            break

        selected.append(best_i)
        selected_set.add(best_i)
        sec = df_stocks.loc[best_i, "sector"]
        sec = sec if isinstance(sec, str) and sec else "Unknown"
        sector_counts[sec] = sector_counts.get(sec, 0) + 1

    out = df_stocks.loc[selected, ["symbol","company_name","sector","i
    out["hybrid_score"] = scores[selected]
    out = out.sort_values("hybrid_score", ascending=False).reset_index
    return out
```

First Random Forest Model Pipeline

In [ ]:

```
```

In [4]:

```
import numpy as np
import pandas as pd
#%pip install yfinance
import yfinance as yf
import warnings
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path
import zipfile
import requests
warnings.filterwarnings("ignore")
```

In [5]:

```
# Load the data from the zip file

## PATH NEEDS TO BE UPDATED TO GITHUB DATA PATH
zip_path2 = "HelpHerInvest/Data/final_dataset_20260224v2.csv.zip"
with zipfile.ZipFile(zip_path2) as z:
    df = pd.read_csv(z.open("final_dataset_20260224v2.csv"))


## ## ## ##

tickers = df["Ticker"].unique()
print(f"Unique tickers in final dataset: {len(tickers)}")
```

```
print("Size of dataset:",
"Rows:",df.shape[0],
"Columns:",df.shape[1])
```

```
Unique tickers in final dataset: 1993
Size of dataset: Rows: 302024 Columns: 18
```

In [6]:
```
# count number of positive fwd_excess_return
print("Number of positive fwd_excess_return:", (df["fwd_excess"] > 0).
print("Number of negative fwd_excess_return:", (df["fwd_excess"] < 0).
print("Number of zero fwd_excess_return:", (df["fwd_excess"] == 0).sum

df = df.copy()

df['y'] = (df['fwd_excess'] > 0).astype(int)
```

```
Number of positive fwd_excess_return: 143948
Number of negative fwd_excess_return: 151730
Number of zero fwd_excess_return: 287
```

In [7]:
```
# Ensure datetime
df['Date'] = pd.to_datetime(df['Date'])

# Find latest month in dataset
max_date = df['Date'].max()

# Compute cutoff
cutoff_date = max_date - pd.DateOffset(months=2)

# Subset
df_last3 = df[df['Date'] > cutoff_date].copy()
df_last3.head()
```

Out[7]:

	Date	Ticker	mom_1m	mom_3m	mom_6m	mom_12m	mom_12m_
296052	2025-12-31	NVDA	0.4632	-0.1615	-0.0187	-0.0007	-(
296053	2025-12-31	GOOGL	-0.2588	0.9727	0.2777	0.0626	
296054	2025-12-31	AAPL	-0.2902	0.1099	0.0543	-0.0705	-(
296055	2025-12-31	MSFT	-0.2136	-0.4135	-0.1202	-0.0553	-(
296056	2025-12-31	AMZN	-0.1489	0.0413	-0.0823	-0.0795	-(

Split the Data 70/15/15

In [8]:
```
def time_split(df, date_col='Date', train_frac=0.7, val_frac=0.15):
```

```
    df = df.sort_values(date_col).copy()
    unique_dates = df[date_col].drop_duplicates().sort_values().to_num

    # retreive the num of rows to create train/val/test split
    n = len(unique_dates)
    train_end = int(n * train_frac)
    val_end = int(n * (train_frac + val_frac))

    train_dates = unique_dates[:train_end]
    val_dates   = unique_dates[train_end:val_end]
    test_dates  = unique_dates[val_end:]

    train = df[df[date_col].isin(train_dates)]
    val   = df[df[date_col].isin(val_dates)]
    test  = df[df[date_col].isin(test_dates)]

    return train, val, test

training_data, validation_data, testing_data = time_split(df)
print("Train size:", training_data.shape)
print("Val size:", validation_data.shape)
print("Test size:", testing_data.shape)
training_data.tail()
```

```
Train size: (193638, 19)
Val size: (52704, 19)
Test size: (55682, 19)
```

Out[8]:

	Date	Ticker	mom_1m	mom_3m	mom_6m	mom_12m	mom_12m_ex
192479	2021-04-30	SSNC	0.2723	0.1806	-0.0387	-0.1580	-0.
192485	2021-04-30	LH	0.0402	0.0731	-0.0345	-0.0866	-0.(
192486	2021-04-30	IHG	-0.0824	0.0408	-0.0311	-0.1020	-0.(
192487	2021-04-30	CHD	-0.6758	-0.6346	-0.0551	-0.1904	-0.1
192484	2021-04-30	NVR	0.3069	-0.0899	-0.0381	-0.0860	-0.(

Preprocess the Data

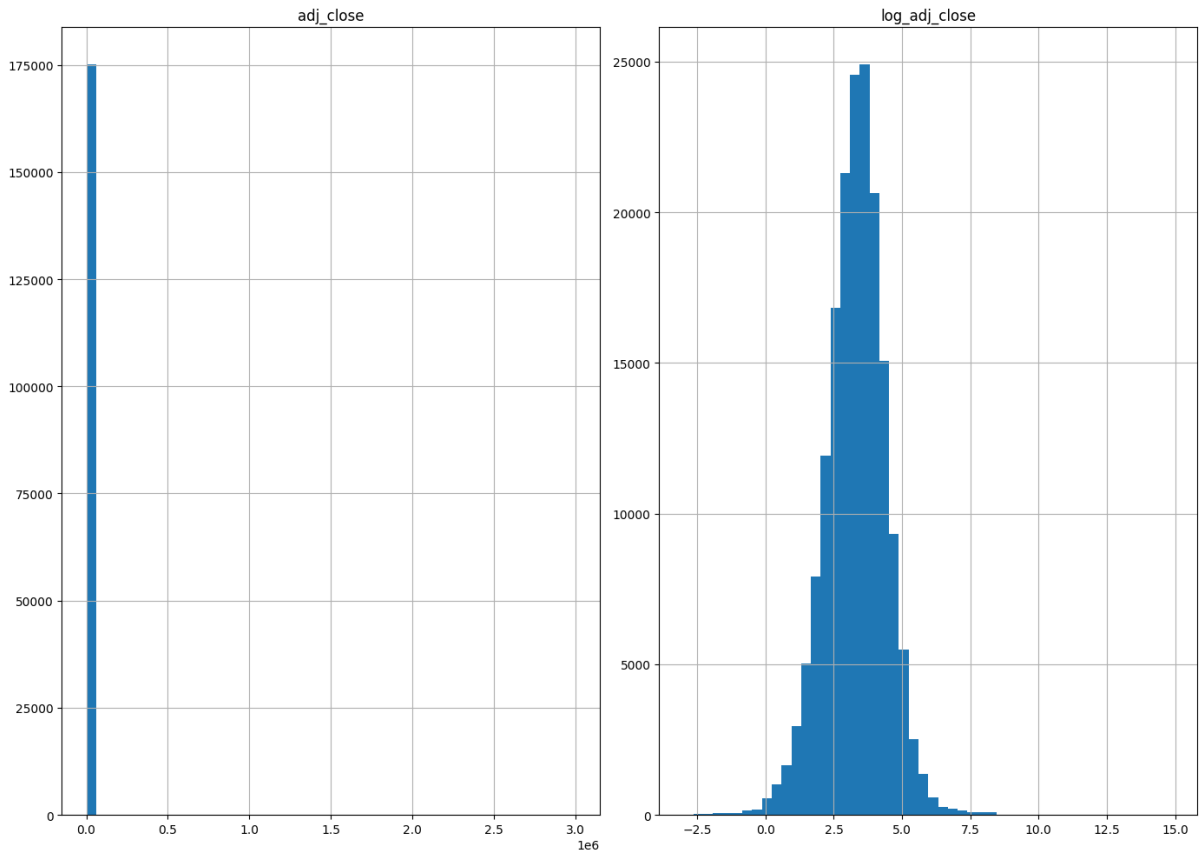
In [9]:

```
def preprocess_data(df):
```

```
    # preprocess the data by dropping rows with missing values
    print("Missing fwd_excess values:")

    df_dropped = df.dropna()
    print("---  ---  ---")
    print("Length of original:",len(df.index))
    print("Length of dropped:",len(df_dropped.index))
    print("Sanity check - missing fwd_excess values after dropping:")
    print(df_dropped[["Date","Ticker","fwd_excess"]].tail(5))



    # Create log_adj_close column
    df_dropped["log_adj_close"] = np.log(df_dropped["adj_close"])
    df_dropped[["adj_close","log_adj_close"]].hist(figsize=(14, 10), b
    plt.tight_layout()
    plt.show()

    return df_dropped
```

In [10]:
```
train_df = preprocess_data(training_data)
train_df.head(5)
```

```
Missing fwd_excess values:
---  ---  ---
Length of original: 193638
Length of dropped: 175057
Sanity check - missing fwd_excess values after dropping:
             Date Ticker  fwd_excess
192479 2021-04-30   SSNC      0.0042
192485 2021-04-30     LH      0.0596
192486 2021-04-30    IHG     -0.1253
192487 2021-04-30    CHD     -0.0415
192484 2021-04-30    NVR     -0.0135
```



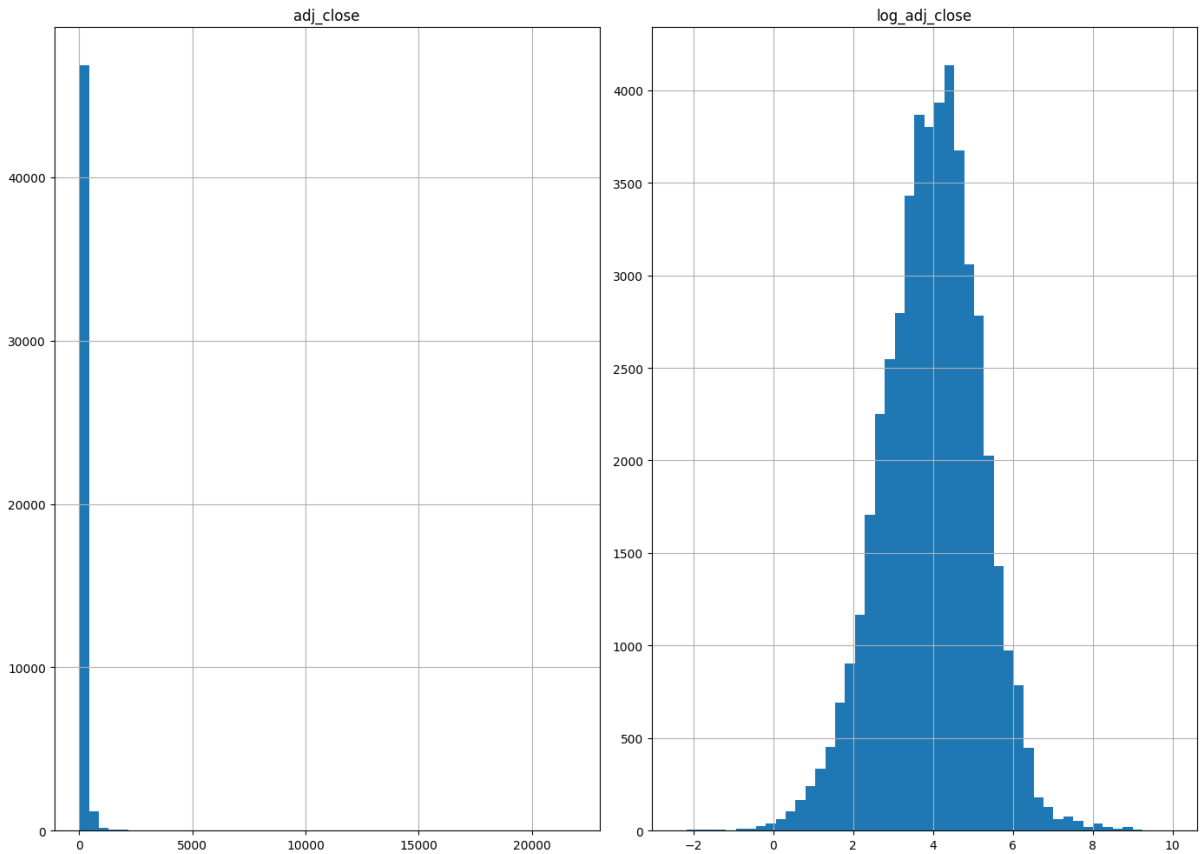
Out[10]:

	Date	Ticker	mom_1m	mom_3m	mom_6m	mom_12m	mom_12m_ex_1
14213	2011-01-31	TTMI	0.4972	2.4448	1.1222	0.4989	0.32
14216	2011-01-31	DSGX	-0.2733	-0.0527	-0.1622	-0.4035	-0.34
14215	2011-01-31	CHDN	-0.6391	0.2684	-0.2432	-0.4573	-0.28
14214	2011-01-31	GSAT	-0.9448	-1.6521	-1.5770	0.1845	0.55
14218	2011-01-31	MASI	0.1469	-0.5690	0.3651	-0.3740	-0.44

In [11]:

```
test_df = preprocess_data(testing_data)
test_df.head(5)
```

```
Missing fwd_excess values:
---  ---  ---
Length of original: 55682
Length of dropped: 48493
Sanity check - missing fwd_excess values after dropping:
            Date Ticker  fwd_excess
294730 2025-11-30    GPN      0.0285
294731 2025-11-30   BURL      0.2126
294732 2025-11-30   KTOS      0.1828
294733 2025-11-30   TRMB     -0.1891
294728 2025-11-30   SBAC      0.0290
```



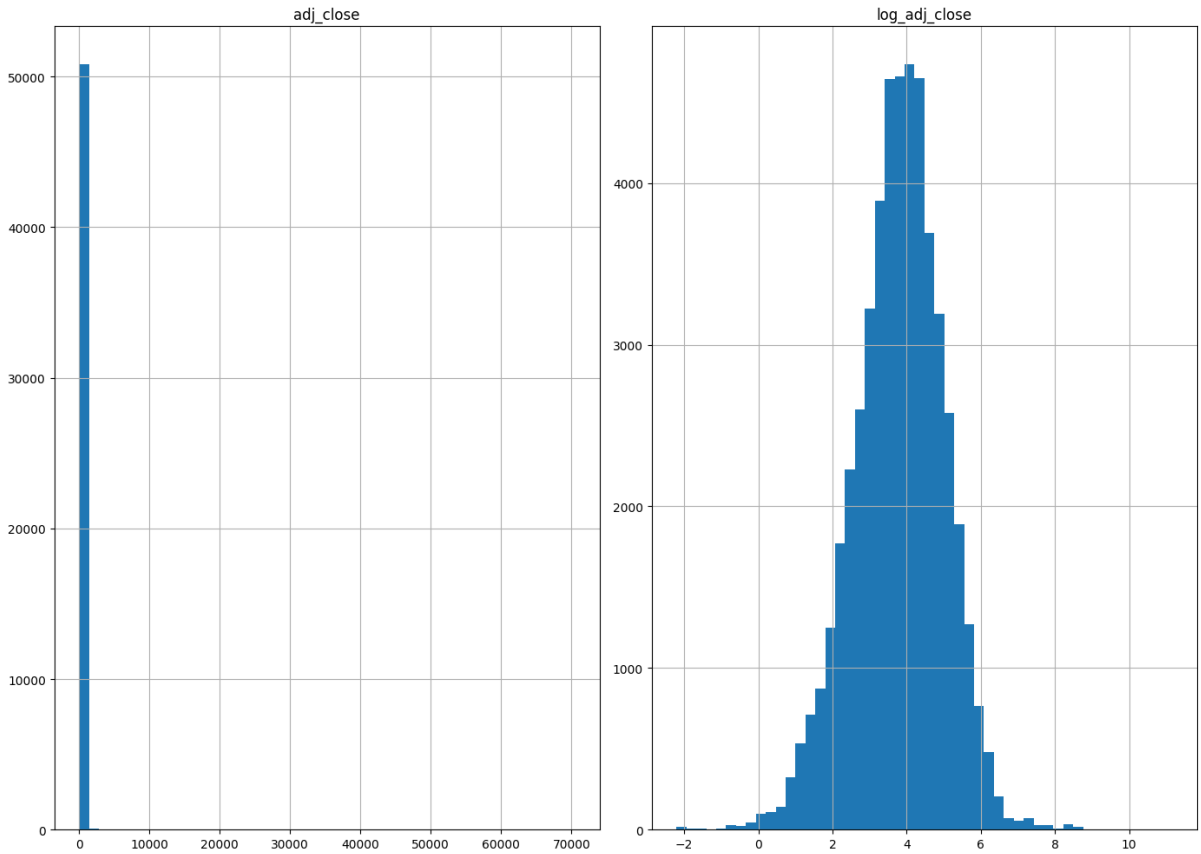
Out[11]:

	Date	Ticker	mom_1m	mom_3m	mom_6m	mom_12m	mom_12m_e
247583	2023-10-31	MNSO	0.2820	1.8365	1.5085	10.5307	10.
247584	2023-10-31	ACT	0.6388	0.7340	0.6076	0.1936	0.
247585	2023-10-31	ZETA	-0.1585	-0.1970	-0.7219	-0.3790	-0.
247586	2023-10-31	RHP	0.7997	0.1445	-0.0652	-0.2143	-0.
247592	2023-10-31	ARW	-0.4508	-0.4682	-0.0130	0.1119	0.

In [12]:
```
val_df = preprocess_data(validation_data)
```

```python
val_df.head(5)
```

```
Missing fwd_excess values:
---  ---  ---
Length of original: 52704
Length of dropped: 50961
Sanity check - missing fwd_excess values after dropping:
            Date Ticker  fwd_excess
245100 2023-09-30    HPQ      0.0652
245099 2023-09-30    PFG     -0.0149
245098 2023-09-30   TWLO      0.1799
245097 2023-09-30   CHRW     -0.1058
245108 2023-09-30    RCI      0.1129
```



Out[12]:

	Date	Ticker	mom_1m	mom_3m	mom_6m	mom_12m	mom_12m_ex
194796	2021-05-31	SITE	-0.6464	-0.0770	-0.0860	-0.0469	-0.
194804	2021-05-31	DLB	-0.6009	-0.5411	-0.3627	-0.0460	-0.
194803	2021-05-31	M	0.8177	0.5672	1.0082	0.2009	0.
194805	2021-05-31	TRNO	-0.3711	0.2303	-0.3603	-0.1159	-0.
194802	2021-05-31	DAR	-0.3743	-0.0735	0.2600	0.2133	0.

In [13]:

```
feature_cols = [col for col in train_df.columns if col not in ["Date",

X_train, y_train = train_df[feature_cols], train_df['y']
X_val, y_val     = val_df[feature_cols],   val_df['y']
X_test, y_test   = test_df[feature_cols],  test_df['y']
```

In [14]:

```
# LightGBM Model 1 with more conservative hyperparameters
%pip install lightgbm
import lightgbm as lgb

model = lgb.LGBMClassifier(
    n_estimators=1000,
    learning_rate=0.01,
    max_depth=6,
    num_leaves=31
)

model.fit(
    X_train,
    y_train,
    eval_set=[(X_val, y_val)],
    #callbacks=[lgb.early_stopping(50)]
)
```

```
Defaulting to user installation because normal site-packages is not wri
teable
Requirement already satisfied: lightgbm in ./.local/lib/python3.12/site
-packages (4.6.0)
Requirement already satisfied: numpy>=1.17.0 in /opt/tljh/user/lib/pyth
on3.12/site-packages (from lightgbm) (2.2.6)
Requirement already satisfied: scipy in /opt/tljh/user/lib/python3.12/s
ite-packages (from lightgbm) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
[LightGBM] [Info] Number of positive: 86255, number of negative: 88802
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead
of testing was 0.028305 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 3825
[LightGBM] [Info] Number of data points in the train set: 175057, numbe
r of used features: 15
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.492725 -> initscore=-
0.029101
[LightGBM] [Info] Start training from score -0.029101
```

Out[14]:

```
▾ LGBMClassifier ⓘ
▸ Parameters
```

In [15]:

```python
# LightGBM Model 2 with more aggressive hyperparameters

model2 = lgb.LGBMClassifier(
    n_estimators=1000,
    learning_rate=0.01,
    max_depth=12,
    num_leaves=31,
    class_weight="balanced",
)

model2.fit(
    X_train,
    y_train,
    eval_set=[(X_val, y_val)],
    #callbacks=[lgb.early_stopping(50)]
)
```

```
[LightGBM] [Info] Number of positive: 86255, number of negative: 88802
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead
of testing was 0.013833 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 3825
[LightGBM] [Info] Number of data points in the train set: 175057, numbe
r of used features: 15
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=-
0.000000
[LightGBM] [Info] Start training from score -0.000000
```

Out[15]:

```
▾ LGBMClassifier ⓘ
▸ Parameters
```

In [16]:

```python
# LightGBM Model 3 with even more aggressive hyperparameters

model3 = lgb.LGBMClassifier(
    n_estimators=500,
    learning_rate=0.01,
    max_depth=6,
    num_leaves=20
)

model3.fit(
    X_train,
    y_train,
    eval_set=[(X_val, y_val)],
    #callbacks=[lgb.early_stopping(50)]
)
```

```
[LightGBM] [Info] Number of positive: 86255, number of negative: 88802
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead
of testing was 0.004329 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 3825
[LightGBM] [Info] Number of data points in the train set: 175057, numbe
r of used features: 15
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.492725 -> initscore=-
0.029101
[LightGBM] [Info] Start training from score -0.029101
```

Out[16]:

```
▾ LGBMClassifier ⓘ
▸ Parameters
```

In [17]:

```python
from sklearn.metrics import f1_score, classification_report, confusion

def evaluate(model, X_test, y_test):
    y_pred = model.predict(X_test)
    probs = model.predict_proba(X_test)[:,1]

    print("Classification Report:\n")
    print(classification_report(y_test, y_pred))
    print("F1:", round(f1_score(y_test, y_pred, average='macro'), 4))
    print("ROC-AUC:", round(roc_auc_score(y_test, probs, average='macr


print("LightGBM Model 1:")
evaluate(model, X_val, y_val)
```

```python
print("\n-----  -----  -----  -----  -----  -----  -----  -----  -----
print("LightGBM Model 2:")
evaluate(model2, X_val, y_val)
print("\n-----  -----  -----  -----  -----  -----  -----  -----  -----
print("LightGBM Model 3:")
evaluate(model3, X_val, y_val)
```

```
LightGBM Model 1:
Classification Report:

              precision    recall  f1-score   support

           0       0.53      0.62      0.57     26843
           1       0.49      0.40      0.44     24118

    accuracy                           0.52     50961
   macro avg       0.51      0.51      0.51     50961
weighted avg       0.51      0.52      0.51     50961

F1: 0.5063
ROC-AUC: 0.5129

-----  -----  -----  -----  -----  -----  -----  -----  -----  -----
LightGBM Model 2:
Classification Report:

              precision    recall  f1-score   support

           0       0.54      0.53      0.53     26843
           1       0.48      0.49      0.48     24118

    accuracy                           0.51     50961
   macro avg       0.51      0.51      0.51     50961
weighted avg       0.51      0.51      0.51     50961

F1: 0.5088
ROC-AUC: 0.5134

-----  -----  -----  -----  -----  -----  -----  -----  -----  -----
LightGBM Model 3:
Classification Report:

              precision    recall  f1-score   support

           0       0.53      0.67      0.59     26843
           1       0.48      0.35      0.41     24118

    accuracy                           0.52     50961
   macro avg       0.51      0.51      0.50     50961
weighted avg       0.51      0.52      0.50     50961

F1: 0.4991
ROC-AUC: 0.5144
```

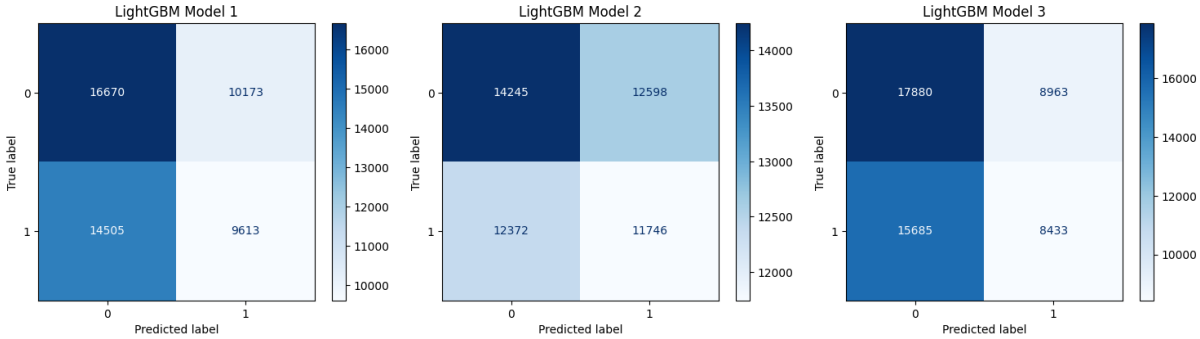
In [18]:
```
def evaluate_to_df(model, X_test, y_test, model_name):
    y_pred = model.predict(X_test)
    probs = model.predict_proba(X_test)[:, 1]

    # Classification report as table
    report_dict = classification_report(y_test, y_pred, output_dict=Tr
    report_df = pd.DataFrame(report_dict).T

    # Add summary metrics
    report_df.loc["overall", "f1_marco"] = f1_score(y_test, y_pred, av
    report_df.loc["overall", "roc_auc"] = roc_auc_score(y_test, probs,
    report_df["model"] = model_name

    return report_df

results = {
    "LightGBM Model 1": evaluate_to_df(model, X_val, y_val, "LightGBM
    "LightGBM Model 2": evaluate_to_df(model2, X_val, y_val, "LightGBM
    "LightGBM Model 3": evaluate_to_df(model3, X_val, y_val, "LightGBM
}

'''
## THIS NEEDS TO BE UPDATED TO GITHUB OUTPUT PATH
class_report_output = r"/Users/christoomey/Desktop/Courses/ADAN8888_Ap

# one sheet per model
with pd.ExcelWriter(class_report_output, engine="openpyxl") as writer:
    for sheet_name, df in results.items():
        df.to_excel(writer, sheet_name=sheet_name)

print(f"Saved to {class_report_output}")
'''
```

Out[18]:
```
'\n## THIS NEEDS TO BE UPDATED TO GITHUB OUTPUT PATH\nclass_report_ou
tput = r"/Users/christoomey/Desktop/Courses/ADAN8888_Applied_Analytic
s_Project/Output/classification_reports_week08.xlsx"\n\n# one sheet p
er model\nwith pd.ExcelWriter(class_report_output, engine="openpyxl")
as writer:\n    for sheet_name, df in results.items():\n        df.to
_excel(writer, sheet_name=sheet_name)\n\nprint(f"Saved to {class_repo
rt_output}")\n'
```

In [19]:
```
from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt

models = {
    "LightGBM Model 1": model,
    "LightGBM Model 2": model2,
    "LightGBM Model 3": model3
}

# Create confusion matrix tables
cm_tables = {}
```

```python
for name, model in models.items():
    y_pred = model.predict(X_val)
    cm = confusion_matrix(y_val, y_pred)  # [[TN, FP], [FN, TP]]
    cm_df = pd.DataFrame(cm,
                         index=["Actual 0", "Actual 1"],
                         columns=["Pred 0", "Pred 1"])
    cm_tables[name] = cm_df
    print(f"\n{name}\n{cm_df}")

# Plot confusion matrix for each model
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for ax, (name, model) in zip(axes, models.items()):
    y_pred = model.predict(X_val)
    ConfusionMatrixDisplay.from_predictions(
        y_val, y_pred, ax=ax, cmap="Blues", values_format="d"
    )
    ax.set_title(name)

plt.tight_layout()
plt.show()
```

```
LightGBM Model 1
          Pred 0  Pred 1
Actual 0   16670   10173
Actual 1   14505    9613

LightGBM Model 2
          Pred 0  Pred 1
Actual 0   14245   12598
Actual 1   12372   11746

LightGBM Model 3
          Pred 0  Pred 1
Actual 0   17880    8963
Actual 1   15685    8433
```



In [ ]:

In [20]:
```python
# Selected the best model from validation performance and evaluate on
print("\n-----  -----  -----  -----  -----  -----  -----  -----  -----
print("LightGBM Model 2 - Test Set:")
evaluate(model2, X_test, y_test)
```

```
-----  -----  -----  -----  -----  -----  -----  -----  -----  -----
LightGBM Model 2 - Test Set:
Classification Report:

              precision    recall  f1-score   support

           0       0.55      0.53      0.54     25892
           1       0.48      0.50      0.49     22601

    accuracy                           0.52     48493
   macro avg       0.52      0.52      0.52     48493
weighted avg       0.52      0.52      0.52     48493

F1: 0.5155
ROC-AUC: 0.5215
```